

ELO329 - Diseño y Programación Orientados a Objetos

Desarrollo en Qt 6

Agustín J. González
Patricio Olivares R.

Universidad Técnica Federico Santa María



Objetivo y ruta

Qt permite construir interfaces gráficas C++ sin programar directamente contra cada sistema operativo.

En esta clase conectamos Qt con temas de ELO329:

- clases, herencia, composición y encapsulamiento.
- eventos, signals y slots.
- interfaz gráfica separada de la lógica.
- dibujo, escenas y animaciones.

La meta no es memorizar widgets. La meta es estructurar una aplicación Qt mantenible.

Qt 6 en este curso

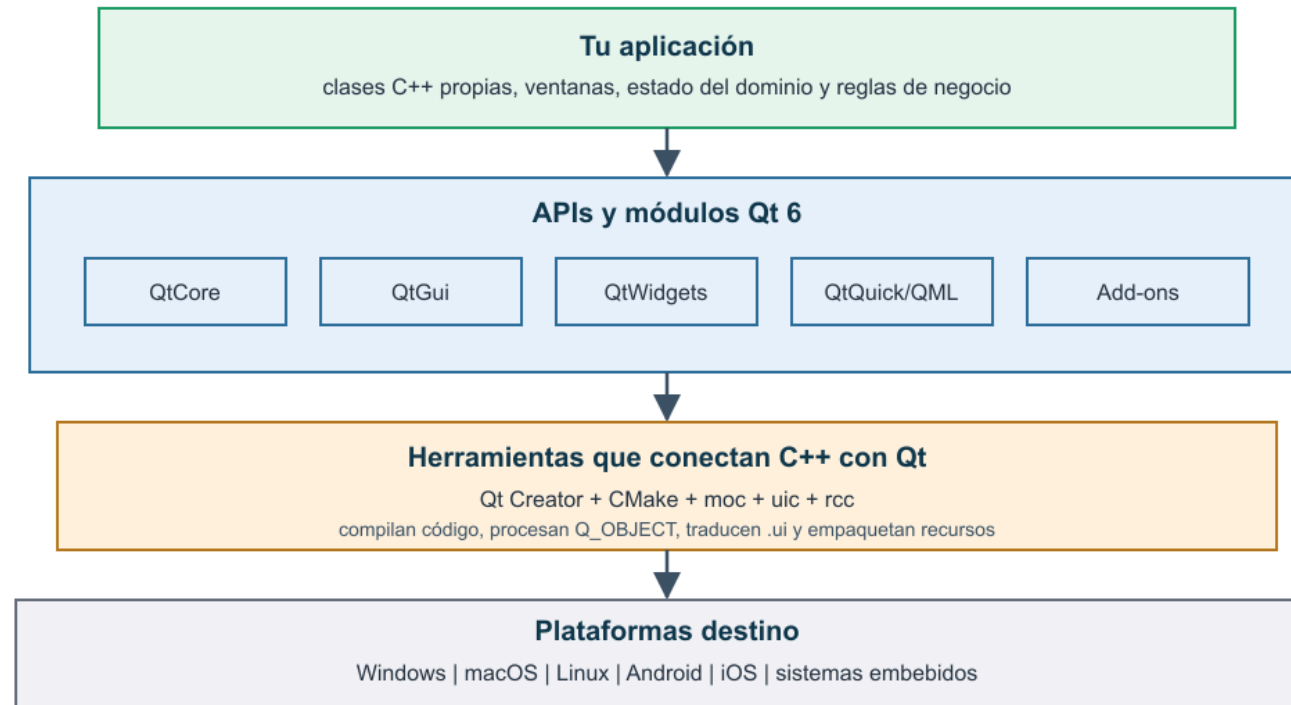
Flujo de trabajo:

- `CMakeLists.txt` para configurar.
- `QApplication` para iniciar el ciclo de eventos.
- Qt Widgets y Qt Designer para la interfaz.
- signals y slots con sintaxis tipada.

`qmake` sigue existiendo. Para trabajos nuevos del curso, preferimos CMake.

Arquitectura general

Qt 6: una capa de abstracción para aplicaciones C++



La aplicación se escribe contra APIs Qt, y Qt abstrae diferencias entre Windows, macOS, Linux y otras plataformas.

Módulos y enfoques de interfaz

Componente	Rol
QtCore	QObject , eventos, archivos, hilos, temporizadores
QtGui	dibujo, fuentes, imágenes, entrada de usuario
QtWidgets	controles de escritorio como QWidget , QMainWindow , QPushButton
QtQuick/QML	interfaces declarativas, fluidas y animadas

En ELO329 partimos con Qt Widgets porque trabaja directamente con C++ y refuerza POO.

Herramientas y proyecto base

Qt Creator integra edición, diseño visual, compilación, ejecución y depuración.

```
MiApp/  
├─ CMakeLists.txt  
├─ main.cpp  
├─ mainwindow.h  
├─ mainwindow.cpp  
├─ mainwindow.ui  
└─ build/
```

`mainwindow.ui` describe la interfaz. `mainwindow.h/.cpp` define el comportamiento.

CMake mínimo

```
cmake_minimum_required(VERSION 3.16)
project>HelloQt LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(Qt6 REQUIRED COMPONENTS Widgets)
qt_standard_project_setup()

qt_add_executable>HelloQt
    main.cpp mainwindow.h mainwindow.cpp mainwindow.ui
)

target_link_libraries>HelloQt PRIVATE Qt6::Widgets)
```

qt_standard_project_setup() activa reglas como AUTOMOC y AUTOUIIC .

main.cpp y ciclo de eventos

```
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    MainWindow window;
    window.show();

    return app.exec();
}
```

`app.exec()` mantiene activa la aplicación esperando eventos.

QObject y ciclo de vida

QObject es la base de gran parte de Qt: organiza objetos, signals/slots, propiedades, eventos y temporizadores.

```
auto *button = new QPushButton("Cerrar", this);  
auto *timer = new QTimer(this);
```

Si un QObject tiene padre, el padre destruye a sus hijos.

Widgets y ventana principal

Un widget puede mostrarse, recibir eventos, contener otros widgets y participar en layouts.

`QMainWindow` entrega menú, barras de herramientas, zona central y barra de estado.

Use `QMainWindow` para herramientas completas. Use `QWidget` para ventanas o formularios simples.

Separar interfaz y lógica

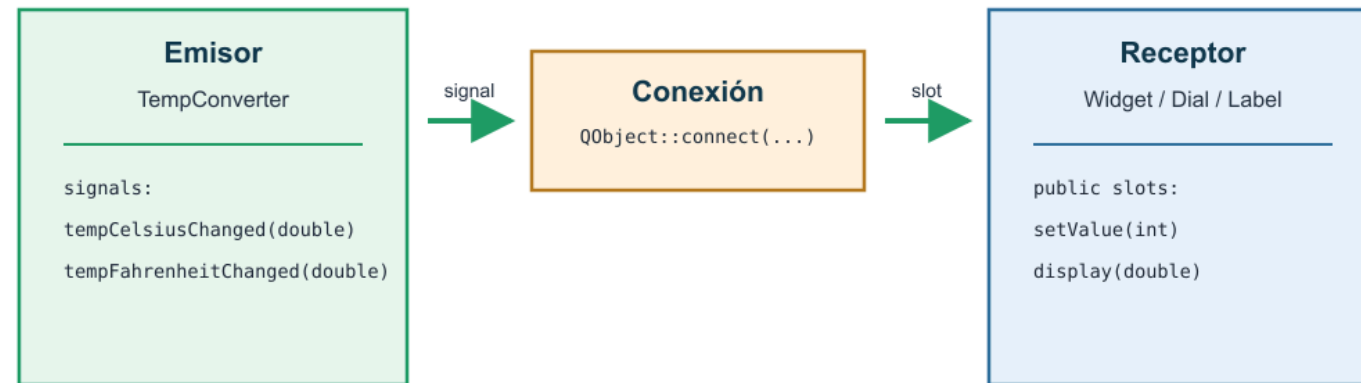
No conviene poner toda la aplicación dentro de `MainWindow`.

Parte	Responsabilidad
<code>MainWindow</code>	leer widgets, actualizar widgets, conectar eventos
Clases de dominio	estado y reglas del problema
Servicios	archivos, red, cálculos, persistencia

La interfaz debe usar el modelo, no convertirse en el modelo.

Signals y slots

Signals y slots: objetos acoplados por contrato, no por dependencia directa



Al emitirse una señal, Qt invoca cada slot conectado con firma compatible.

Propiedad clave para diseño OO

El emisor no necesita saber cuántos receptores existen ni qué harán.

La firma de la señal y del slot define el contrato.

Un signal anuncia que algo ocurrió. Un slot reacciona sin que el emisor conozca directamente a sus receptores.

Declarar y emitir señales

```
class Counter : public QObject {
    Q_OBJECT

public slots:
    void setValue(int value);

signals:
    void valueChanged(int newValue);

private:
    int m_value = 0;
};

void Counter::setValue(int value)
{
    if (value == m_value)
        return;

    m_value = value;
    emit valueChanged(m_value);
}
```

Declarar y emitir señales

`Q_OBJECT` permite que `moc` genere el código auxiliar.

Conectar objetos

```
Counter a;  
Counter b;  
  
QObject::connect(&a, &Counter::valueChanged,  
                &b, &Counter::setValue);  
  
a.setValue(12);
```

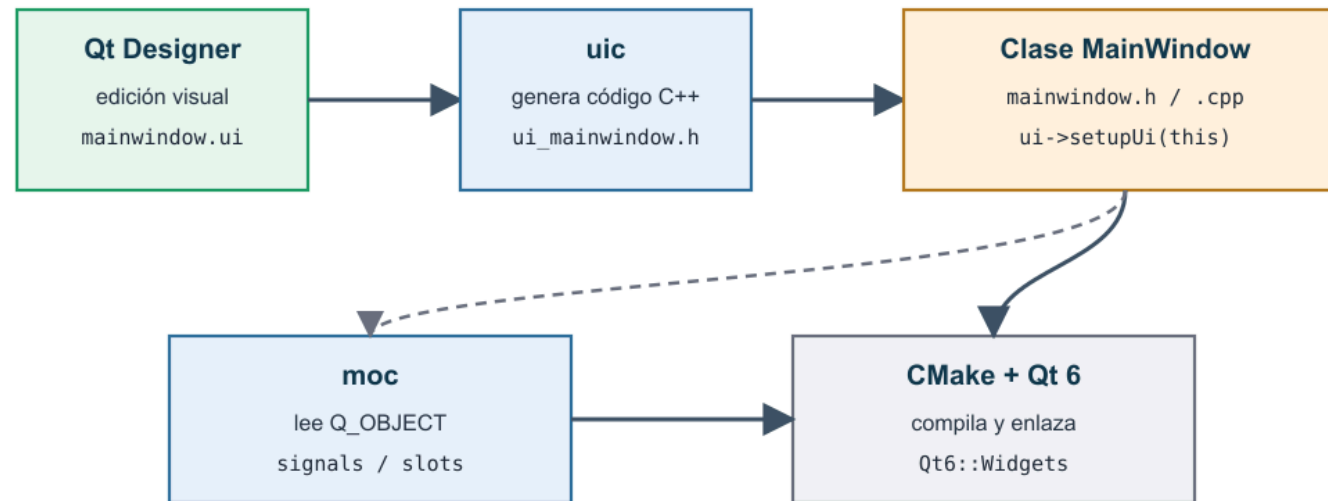
Para widgets:

```
connect(ui->btnSaludar, &QPushButton::clicked, this, [this]() {  
    ui->lblMensaje->setText("Hola desde Qt 6");  
});
```

La sintaxis moderna permite detectar incompatibilidades de tipos en compilación.

Qt Designer y archivos .ui

De Qt Designer a una ventana ejecutable



El archivo .ui describe apariencia; la clase C++ decide comportamiento.

Designer define apariencia y jerarquía de widgets. C++ define comportamiento.

MainWindow con Designer

```
MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    connect(ui->btnSaludar, &QPushButton::clicked,
            this, &MainWindow::saludar);
}

MainWindow::~MainWindow()
{
    delete ui;
}
```

`setupUi()` crea los widgets definidos en `mainwindow.ui`.

Nombres y layouts

El `objectName` definido para cada Widget agregado permite acceder a este desde C++:

```
ui->lblMensaje->setText("Listo");  
ui->btnGuardar->setEnabled(true);
```

Use nombres claros:

Evitar	Preferir
<code>pushButton</code>	<code>btnGuardar</code>
<code>label_2</code>	<code>lblEstado</code>

Use layouts, no coordenadas fijas, para que la interfaz escale bien.

Dibujo en Qt

Técnica	Uso típico
QPainter en paintEvent()	dibujo directo sobre un widget
QGraphicsView + QGraphicsScene	escena con varios objetos 2D

```
void Widget::paintEvent(QPaintEvent *)
{
    QPainter painter(this);
    painter.setPen(Qt::blue);
    painter.drawRect(10, 10, 120, 40);
}
```

Si necesita repintar, use `update()`.

QGraphicsView y QGraphicsScene

```
auto *scene = new QGraphicsScene(this);

scene->addRect(10, 10, 100, 30,
              QPen(Qt::blue),
              QBrush(Qt::cyan));

ui->graphicsView->setScene(scene);
```

`QGraphicsScene` administra items. `QGraphicsView` muestra una vista de la escena.

Es una buena base para simuladores gráficos con varios objetos visibles.

Animaciones con QPropertyAnimation

`QPropertyAnimation` cambia una propiedad Qt a lo largo del tiempo.

En JavaFX animábamos propiedades de nodos. En Qt la idea es la misma: cambiar propiedades visuales, como posición, opacidad o rotación.

```
auto *anim = new QPropertyAnimation(objeto, "pos");

anim->setDuration(2000);
anim->setStartValue(QPointF(100, 100));
anim->setEndValue(QPointF(400, 250));
anim->setEasingCurve(QEasingCurve::InOutQuad);

anim->start();
```

La propiedad debe existir en el sistema de metaobjetos de Qt.

Ejemplo: QPropertyAnimationCircle

En `examples/QPropertyAnimationCircle` :

- `AnimatedRect` hereda de `QGraphicsObject` , por eso puede animar `"pos"` .
- `boundingRect()` define el área del item; `paint()` solo dibuja.
- `QPropertyAnimation(rect, "pos", rect)` anima la posición y usa `rect` como padre.
- `setLoopCount(-1)` repite indefinidamente.
- `QEasingCurve::Linear` mantiene velocidad constante.

Los puntos del círculo se entregan con `setKeyValueAt(t, point)` . Qt interpola entre ellos y el `event loop` actualiza la escena.

Escena con objetos animados

Para un simulador gráfico genérico:

- `QGraphicsView` muestra la escena.
- `QGraphicsScene` contiene el fondo y los objetos.
- `QGraphicsObject` representa cada objeto animado.
- `QPropertyAnimation` mueve un item hacia otro estado.

```
void MovingItem::moveTo(const QPointF &dest)
{
    auto *anim = new QPropertyAnimation(this, "pos");
    anim->setDuration(1500);
    anim->setStartValue(pos());
    anim->setEndValue(dest);
    anim->setEasingCurve(QEasingCurve::InOutQuad);
    anim->start(QAbstractAnimation::DeleteWhenStopped);
}
```

QPropertyAnimation o QTimer

Use `QPropertyAnimation` cuando el movimiento sea una transición entre estados.

```
item->moveTo(QPointF(400, 250));
```

Use `QTimer` cuando necesite una simulación continua paso a paso:

- física propia.
- colisiones.
- velocidad variable.
- actualización manual por frame.

En ambos casos, la interfaz debe mostrar el estado del modelo, no reemplazarlo.

Recursos y flujo recomendado

Qt puede empaquetar recursos con `.qrc` :

- iconos, imágenes, QML, hojas de estilo o datos.
- acceso con rutas como `(":/icons/save.svg"` .
- inclusión desde CMake como fuentes o con `qt_add_resources` .

1. Crear proyecto `Qt Widgets Application` con CMake.
2. Definir clases de dominio.
3. Diseñar la interfaz con layouts.
4. Nombrar widgets con `objectName` claros.
5. Conectar signals y slots en C++.
6. Probar interacciones pequeñas.
7. Mover cálculos fuera de `MainWindow` .

Errores frecuentes

Síntoma	Causa probable
<code>undefined reference to vtable</code>	falta <code>Q_OBJECT</code> , <code>moc</code> no corrió, o CMake no ve el archivo
No se ven cambios en la interfaz	falta <code>setupUi(this)</code> o se modifica otro objeto
Widgets se amontonan	faltan layouts
La aplicación se cierra	falta <code>app.exec()</code>
Doble liberación	mezcla incorrecta de padres Qt y objetos locales

Referencias

- Qt Widgets: <https://doc.qt.io/qt-6/qtwidgets-index.html>
- Signals & Slots: <https://doc.qt.io/qt-6/signalsandslots.html>
- Build with CMake: <https://doc.qt.io/qt-6/cmake-get-started.html>
- Object Trees & Ownership: <https://doc.qt.io/qt-6/objecttrees.html>
- Qt Quick: <https://doc.qt.io/qt-6/qtquick-index.html>
- Material TEL102: 06_QtLibrary